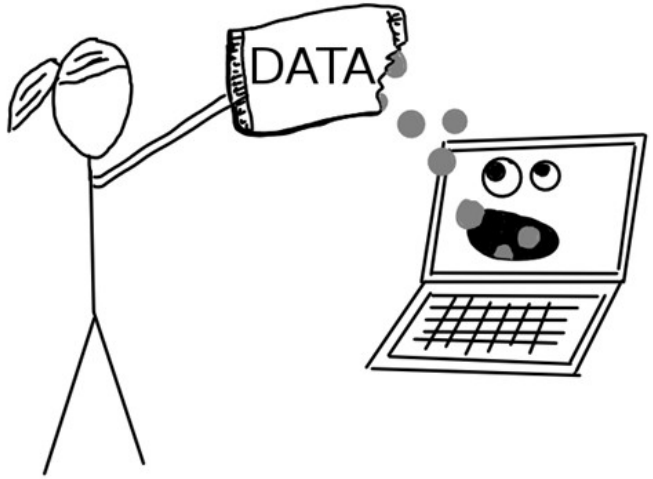
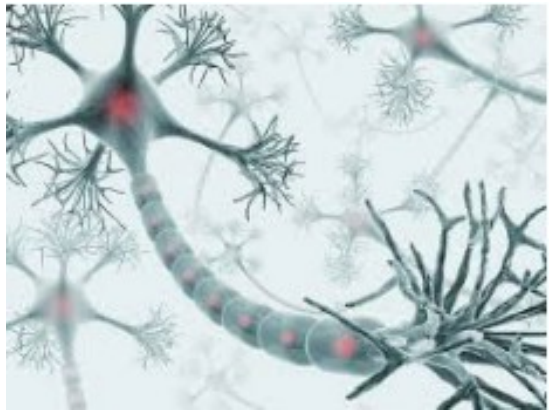
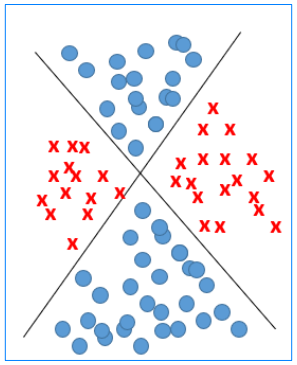






Multilayer Neural Network
non linearly separable problem

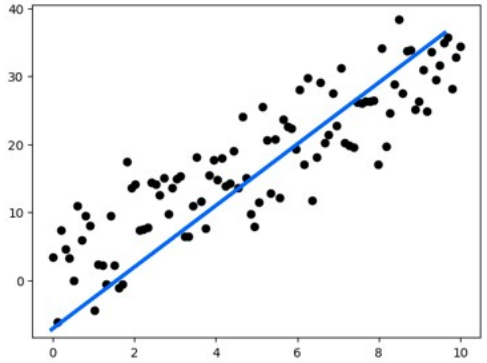
2023/2024

Abdelwahab Naji - Enseignant chercheur
Abdelwahab.naji@gmail.com
ENSAO, Université Mohamed premier - Oujda
https://www.youtube.com/c/AbdelwahabNaji

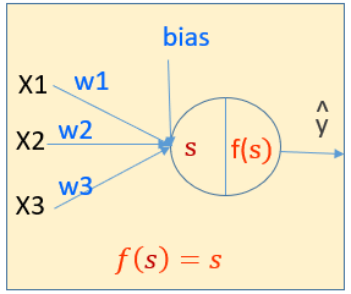
Neural Network for Supervised Learning

1

Simple Linear Regression Problem



Random point cloud

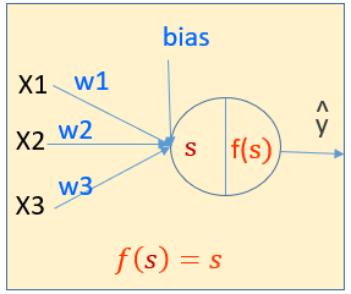


2

Multi Variate Linear Regression Problem

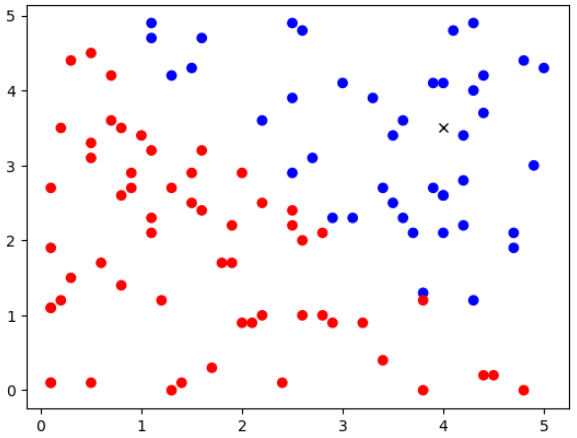
TV	Radio	Newspaper	CA
230.1	37.8	69.2	**22,1**
44.5	39.3	45.1	**10,4**
17.2	45.9	69.3	**9,3**
...			
199.8	2.6	21.2	**10,6**

Advertising p

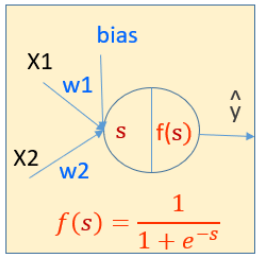


3

Linearly Separable problem

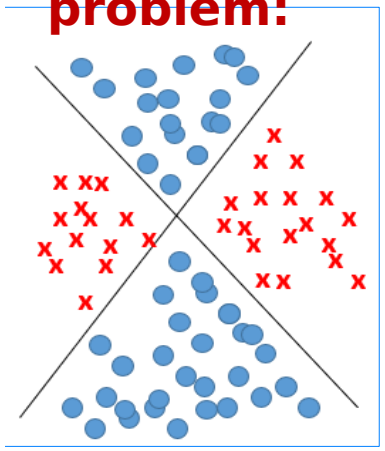


Red or Blue?

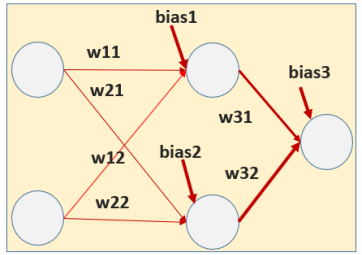


4

Non Linearly Separable problem:

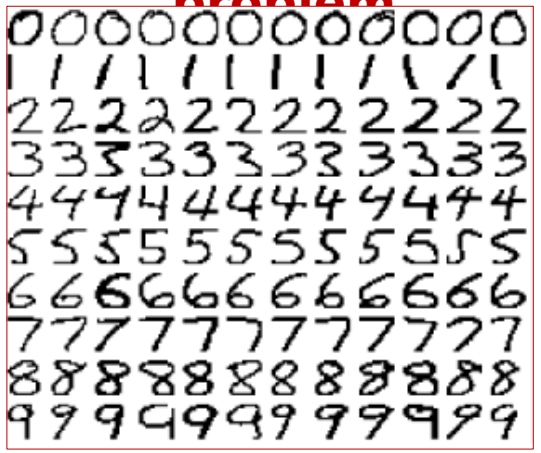


XOR Problem

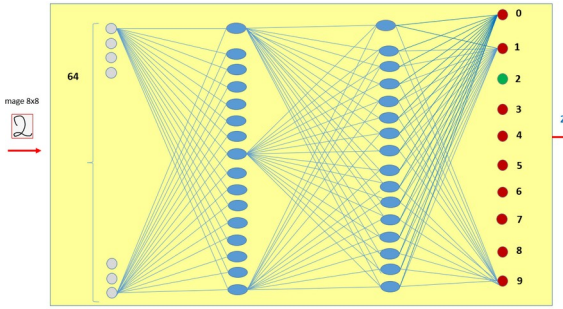


5

Complex classification problem

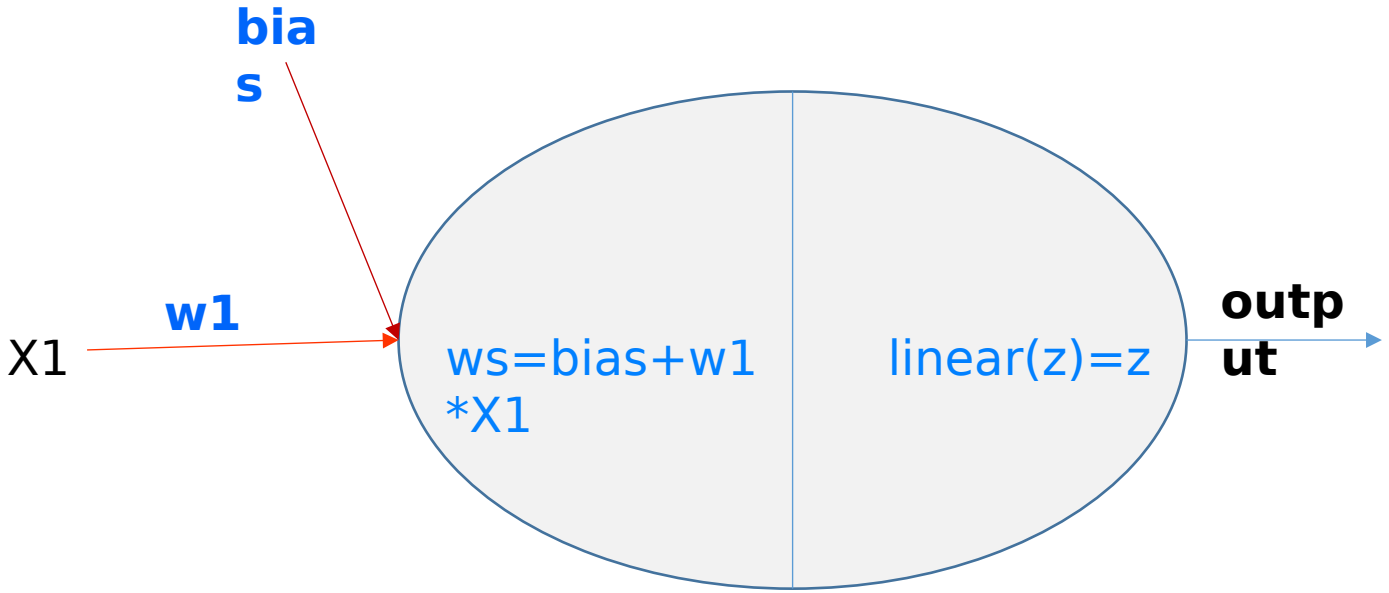


Hand Written digits



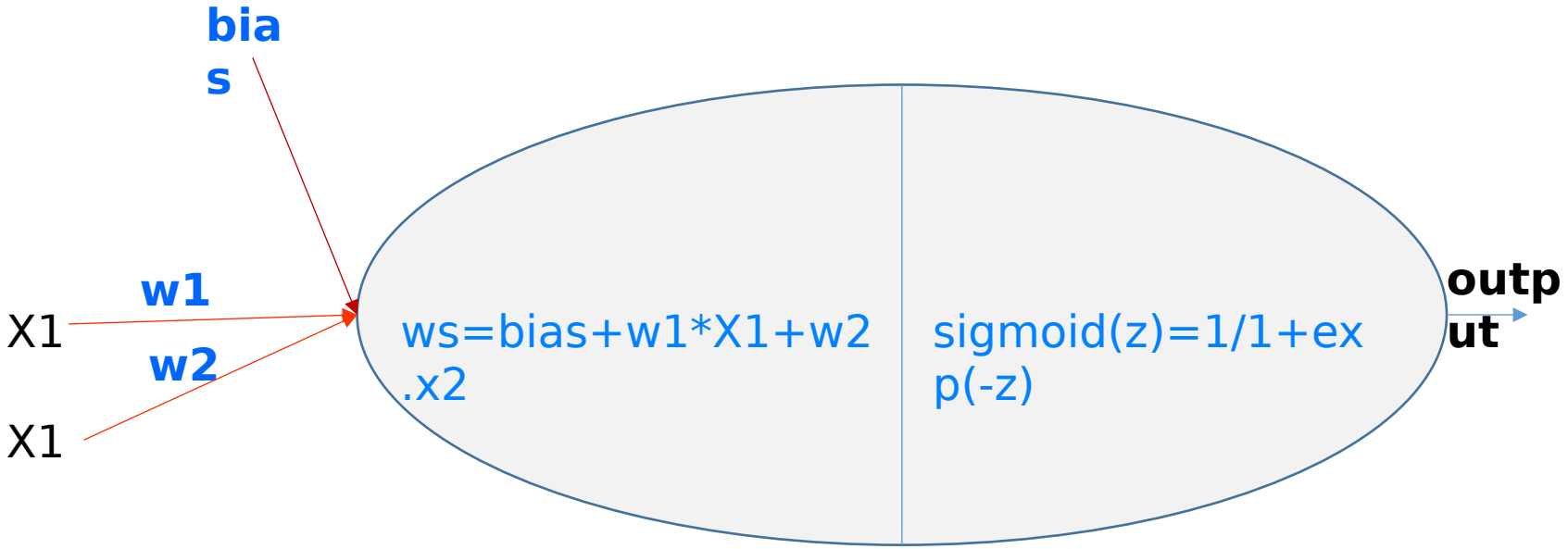
Neural Network – One Neurone

Simple Regression Problem



Neural Network – One Neurone

Simple Classification Problem



1

Simple Regression problem

```
#Librairies
import matplotlib.pyplot as plt
from random import seed
from random import
```

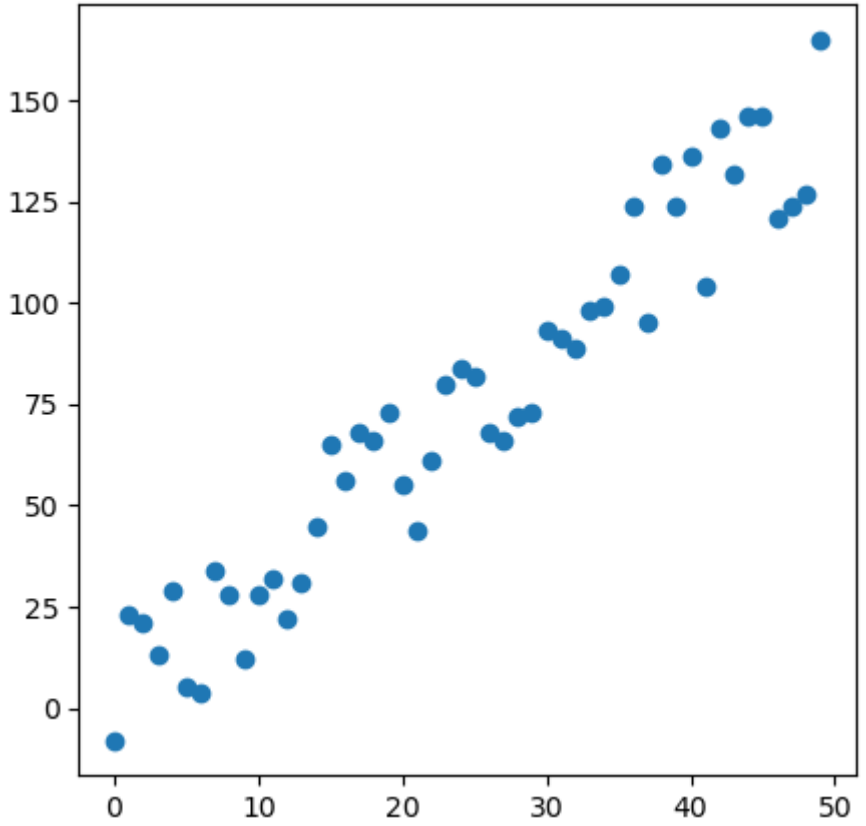
```
np.random.seed(0)
```

```
def f(X):
    return 1+3*X
```

```
def generateData (n):
    X=np.array([i for i in range(n)])
    y_model=f(X)
    y =np.array([i+randrange(-20,20) for i in y_model])
    return X,y_model,y
```

```
n_samples = 50
X,y_model,y=generateData(n_samples)
```

```
plt.figure(figsize=(5, 5))
plt.scatter(X, y)
plt.show()
```



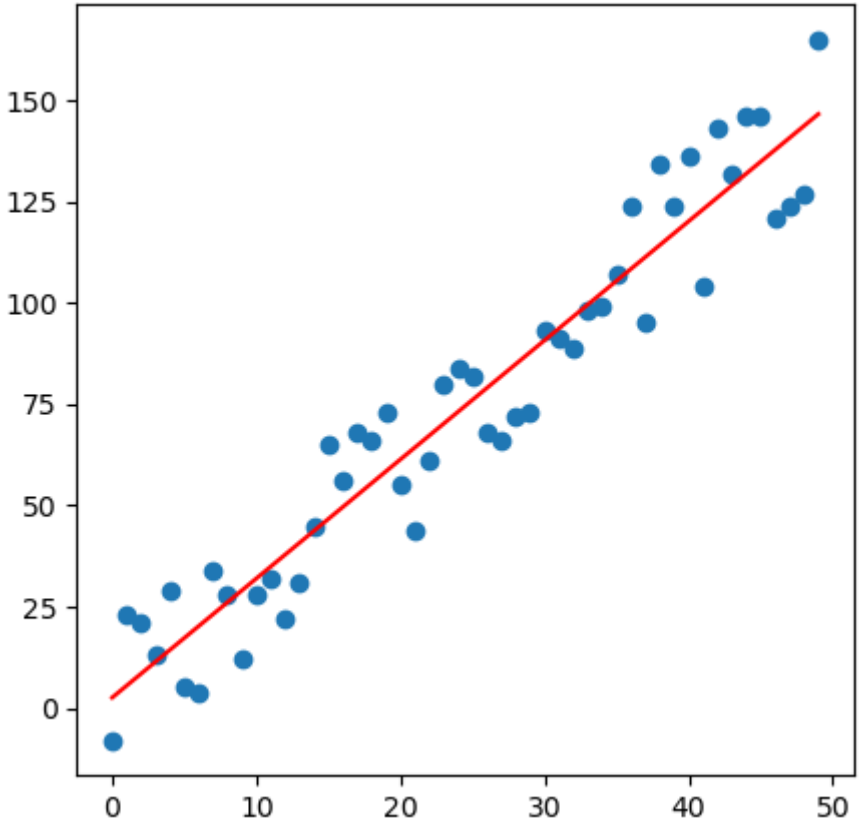
```
def fit(X,y):
    lr=0.001
    epochs=10000
    bias=0.5
    w1=0.5
    n=len(y)
    for i in range(epochs):
        py=bias+w1*X
        dbias=-(2/n)*sum(y-py)
        print(epochs,"---------------->",w1," , ",bias)
        dw1=-(2/n)*sum(X*(y-py))
        w1=w1-dw1*lr
        bias=bias-dbias*lr
    return bias,w1
```

```
bias,w1=fit(X, y)
```

```
def predict(X):
    return bias+w1*X
```

```
predicted_y=predict(X)
```

```
plt.figure(figsize=(5, 5))
plt.title("Simple Regression Model)", size=16)
plt.scatter(X, y)
plt.plot(X, predicted_y, c="red")
plt.show()
```



2

Simple classification problem Linearly Separable

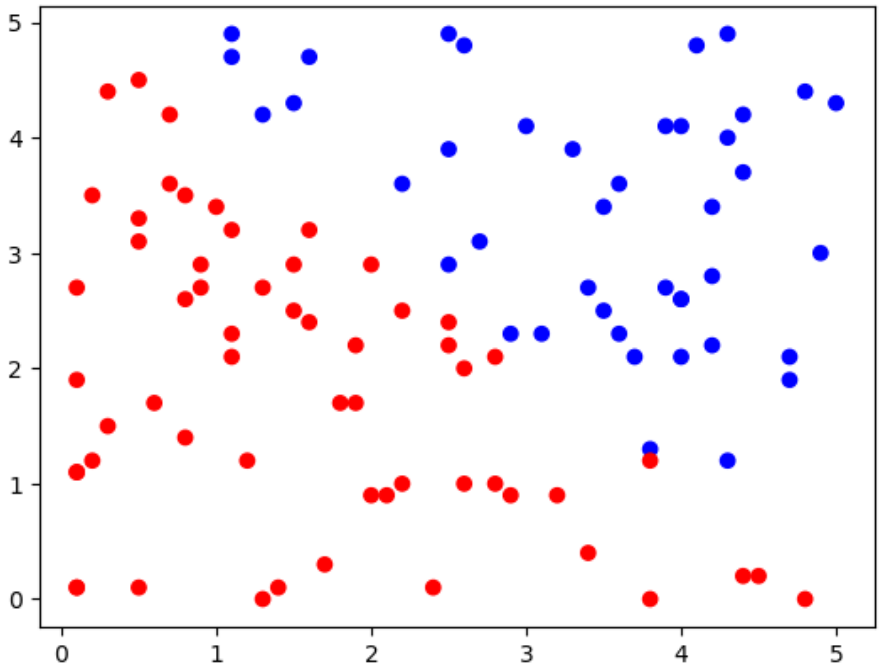
```
import matplotlib.pyplot as plt
import numpy as np
import matplotlib.colors as clr
```

```
def f(x):
    return -x+5
```

```
# generate random floating point values
from random import seed
from random import random
# seed random number generator
def generateData(n):
    seed(1)
    min=0
    max=5
    # generate random numbers between
0-1
    L=[]
    for _ in range(n):
        valueX = random()
        scaledvalueX = min + (valueX *
(max - min))
        valueY = random()
        scaledvalueY = min + (valueY *
(max - min))
        x=round(scaledvalueX,1)
        y=round(scaledvalueY,1)
        y0=f(x)
        cl=0
        if(y>y0):
            cl=1
```

```
L=generateData(10
0)
L=np.array(L)
X=L[:,0]
y=L[:,1]
```

```
colors = ['red','blue']
plt.scatter(L[:,0],L[:,1],c=L[:,2],cmap=clr.ListedColor
map(colors))
```



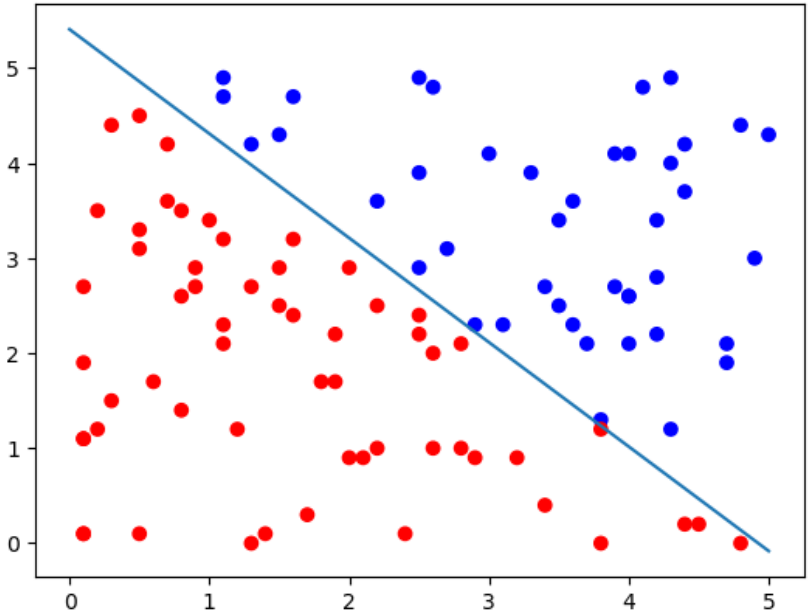
```
def fit(X,y):
    #initialisation
    n=len(y)
    bias=0.5
    w1=0.5
    w2=0.5
    epochs=5000#hyperparamter
    lr=0.01 #hyperparameter
    #learning
    for i in range(epochs):
        y=L[:,2]
        ws=bias+w1*L[:,0]+w2*L[:,1]
        py=sigmoid(ws)
        #update
        dbias=-(2/n)*sum(y-py)
        dw1=-(2/n)*sum(L[:,0]*(y-py))
        dw2=-(2/n)*sum(L[:,1]*(y-py))

        bias=bias-dbias*lr
        w1=w1-dw1*lr
        w2=w2-dw2*lr
    return bias,w1,w2
```

```
bias,w1,w2=fit(X,y)
```

```
def getY(x,bias,w1,w2):
    return -w1/w2*x-bias/w2
```

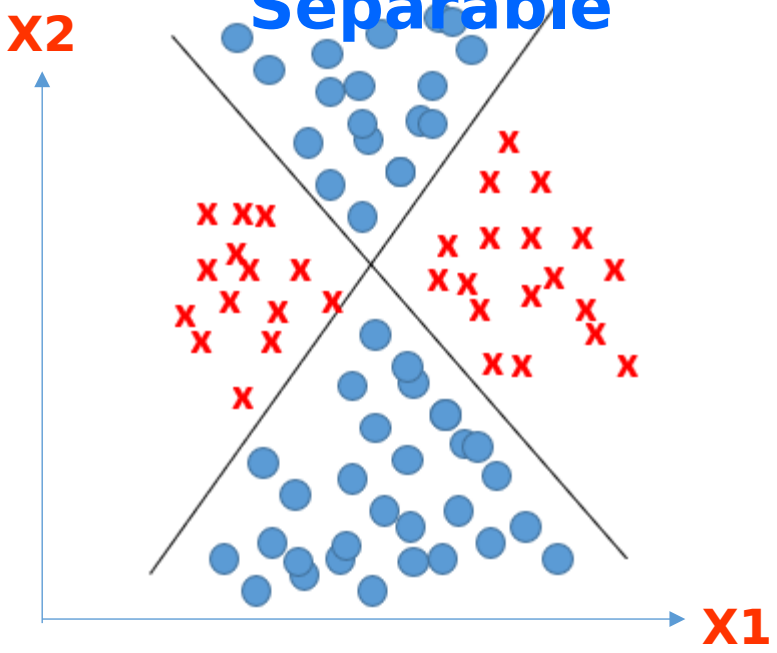
```
x1=0
x2=5
y1=getY(x1,bias,w1,w2)
y2=getY(x2,bias,w1,w2)
```

```
colors = ['red','blue']
plt.scatter(L[:,0],L[:,1],c=L[:,2],cmap=clr.ListedCol
ormap(colors))
#plt.plot([x1,x2],[y1,y2])
plt.plot([x1,x2],[y1,y2])
```



3

Complex classification problem non Linearly Separable



4

Images classification

Mise en œuvre de Neural Network

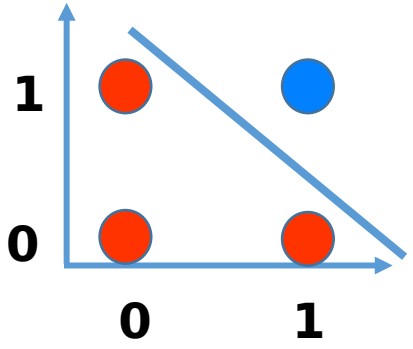
- **NN: Multi – Layer**
  - Présentation du problème **non linéairement séparable**
  - Implémentation du Problème **non linéairement séparable (**avec tensorflow, keras **)**
    - Exemple 1: simple XOR problem
      - Générer les données
      - Créer et entrainer un multilayer neuralnetwork
    - Exemple 2: general XOR problem
      - Générer les données
      - Créer et entrainer un multilayer neuralnetwork
    - Exemple 3: fashion classification problem
    - Exemple 4: hand written digits classification problem
  - Implémentation from scratch
    - Exemple: fashion classification problem

Problem

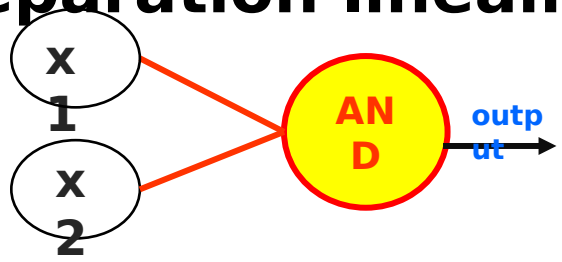
Séparation de
AND, OR,NAND, NOR, XOR

And

	F	T
T	F	T
F	F	F

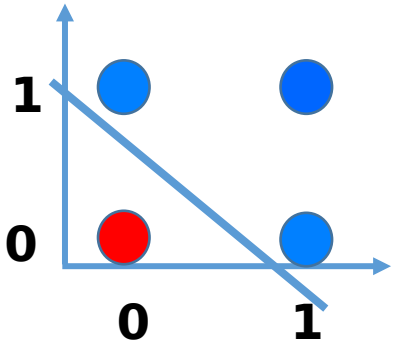


On peut avoir une séparation linéaire

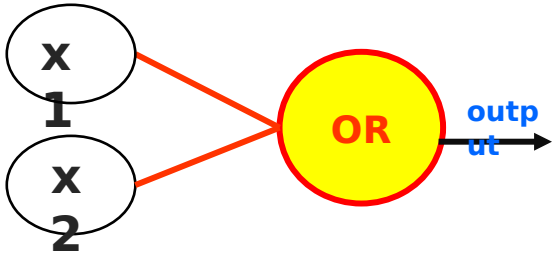


OR

	F	T
T	T	T
F	F	T

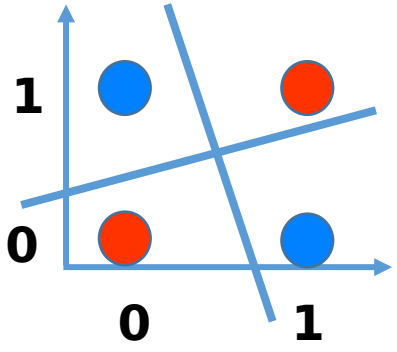


On peut avoir une séparation linéaire



XOR

	F	T
T	T	F
F	F	T

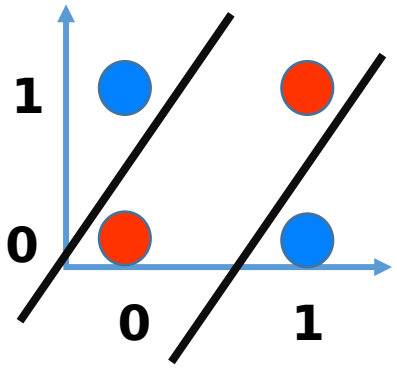


On ne peut pas avoir une séparation linéaire

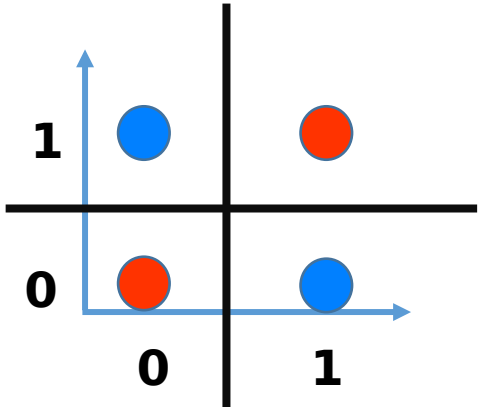
Séparation de X0R

XO
R

	F	T
T	T	F
F	F	T

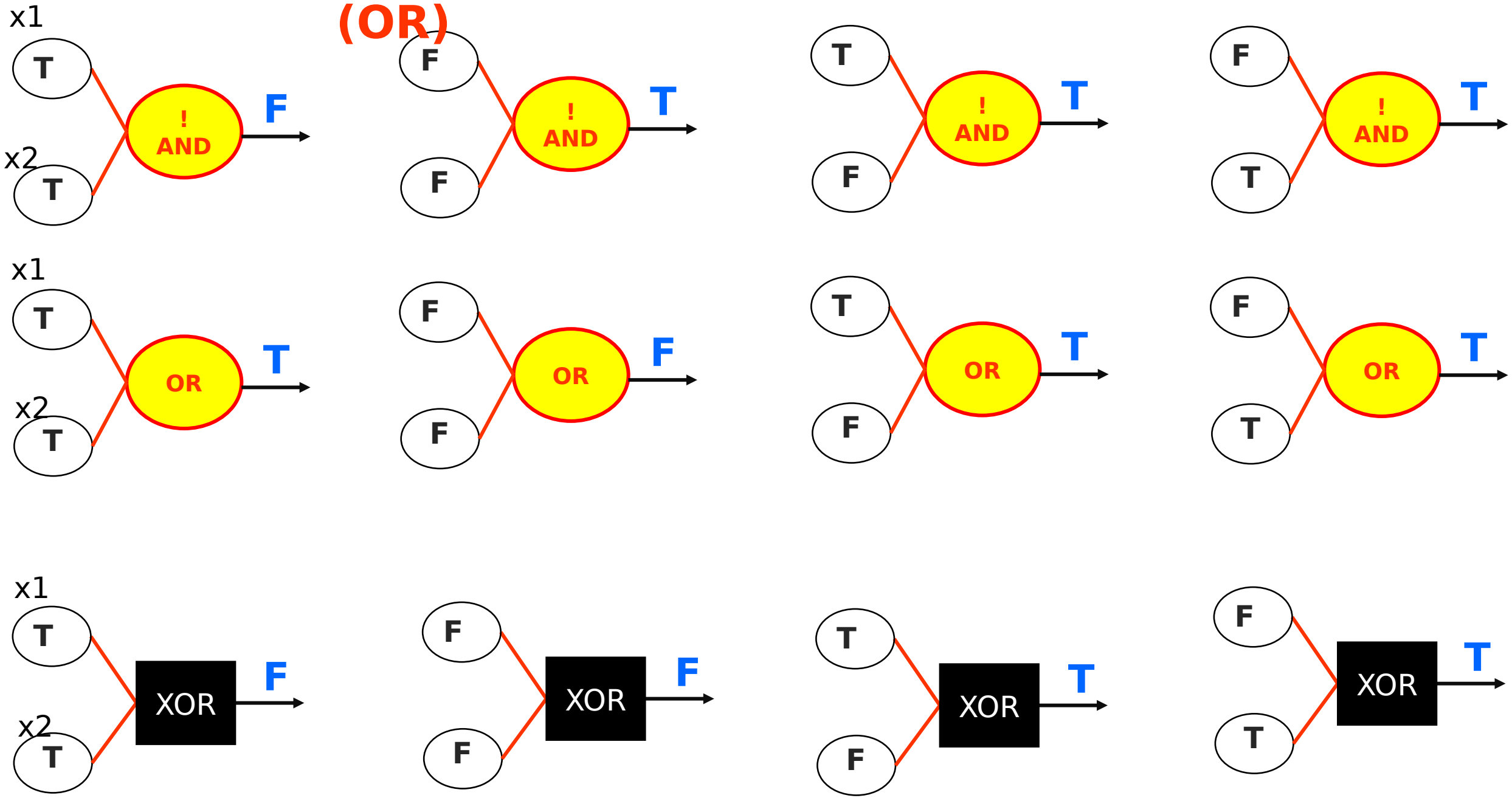


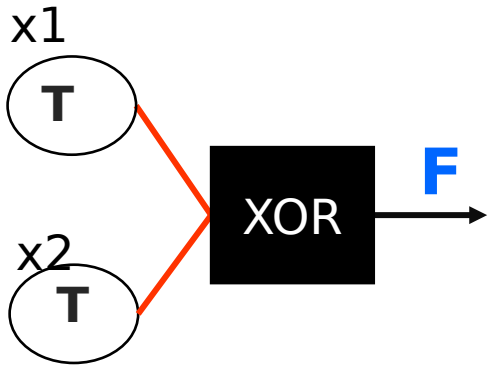
On peut utiliser plus qu'une ligne pour faire la séparation

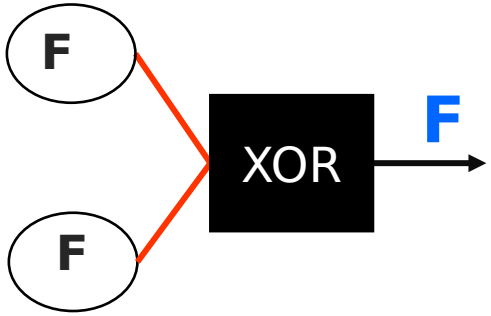


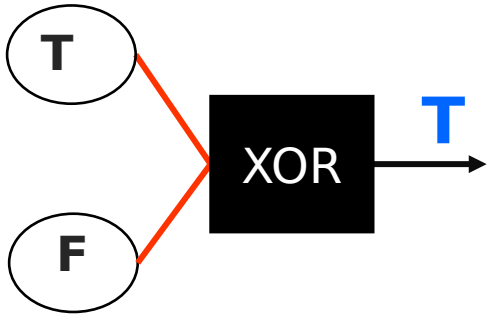
XOR = (!AND) and (OR)

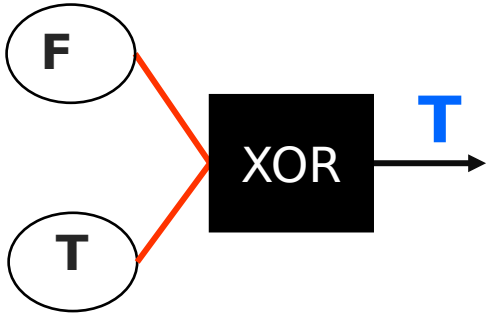
XOR = (!AND) and (OR)

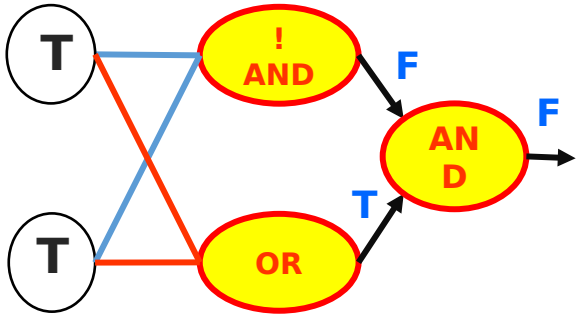


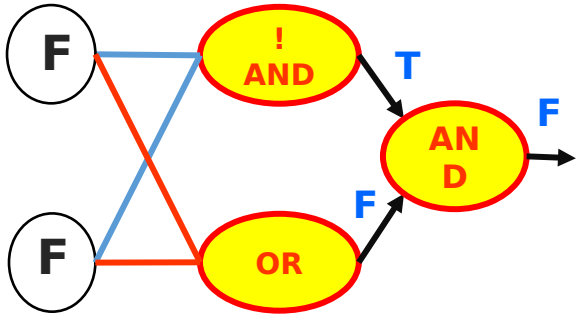

x1
T
x2
T
XOR
F

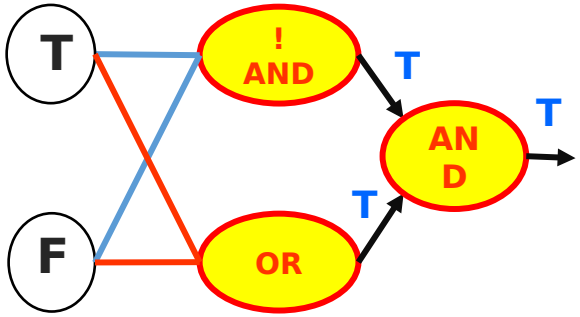

F
F
XOR
F

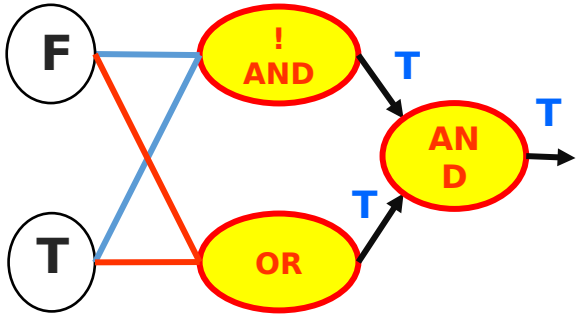

T
F
XOR
T

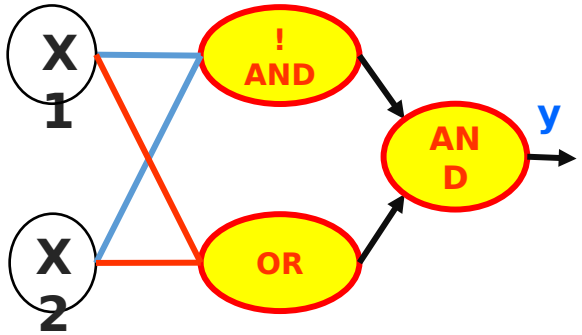

F
T
XOR
T


T
T
! AND
OR
F
T
AN D
F


F
F
! AND
OR
T
F
AN D
F


T
F
! AND
OR
T
T
AN D
T


F
T
! AND
OR
T
T
AN D
T


X 1
X 2
! AND
OR
AN D
y

Utiliser keras pour créer un réseau de neurone avec plusieurs neurones (couche, layer)

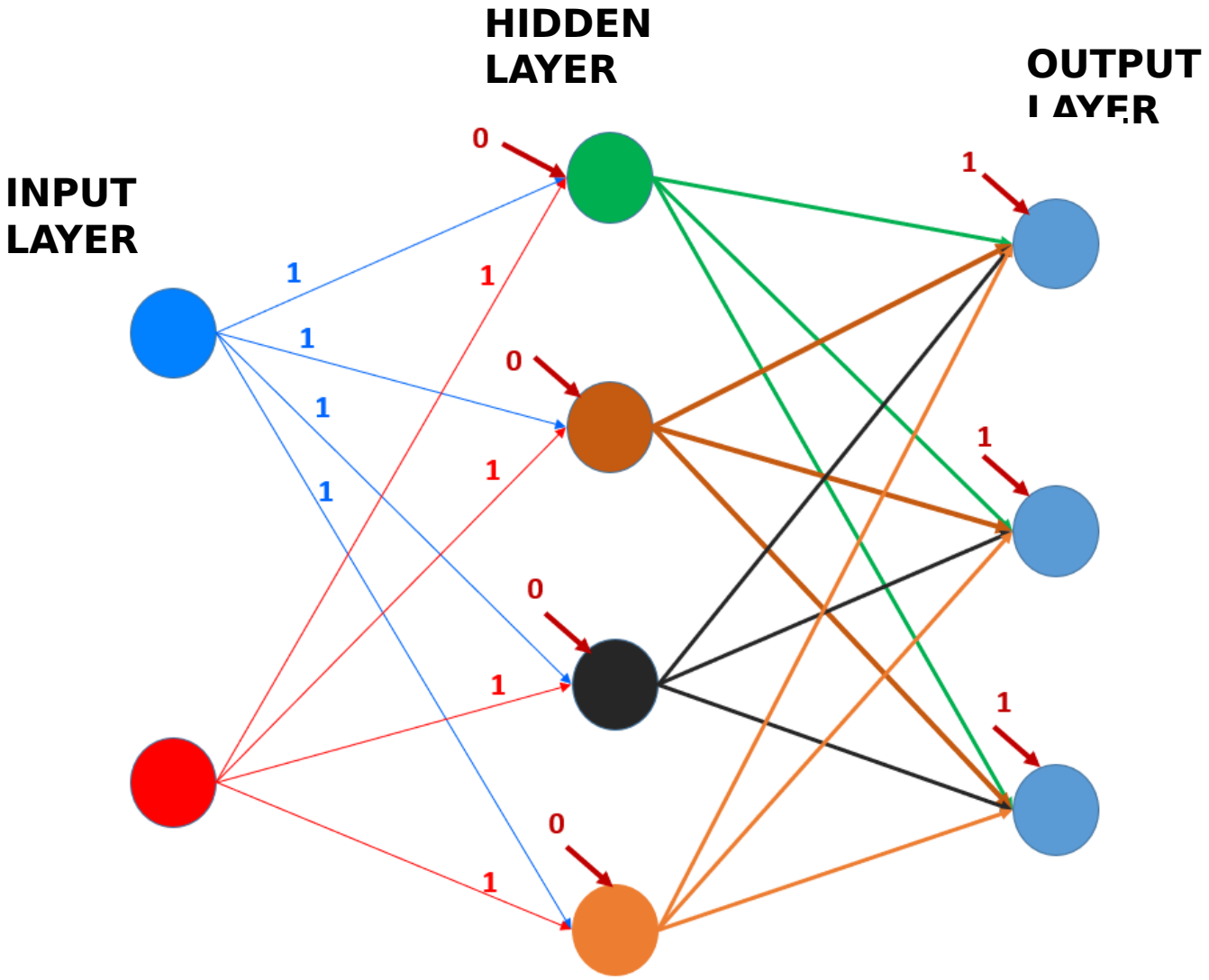
Ce qu’il faut connaitre:

1. **Comment créer un multilayer neural network (en général)?**
2. Découvrir les éléments du neural network créé: coeficients, bias

Neural Network Model et ses paramètres

1. Architecture
2. Créer le modèle
3. Créer une couche et l’ajouter au modèle
4. Paramètres d’une couche

Réseau de neurones de trois couches



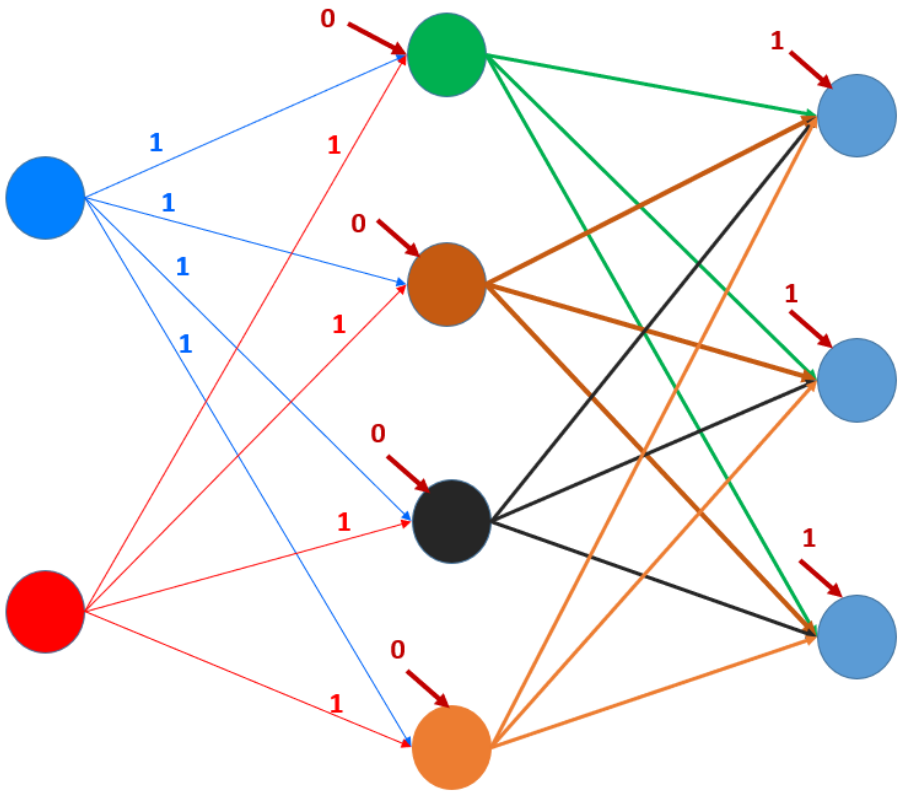
Model et ses paramètres

```
from keras.models import Sequential
from keras.layers import Flatten, Dense, Activation
```

```
model = Sequential()
```

```
#input_layer=Flatten(input_shape=[2, 1])
#model.add(input_layer)
hidden_layer=Dense(4,
input_dim=2,kernel_initializer='ones',bias_initializer='zeros
')
model.add(hidden_layer)
output_layer=Dense(3,
kernel_initializer='random_uniform',bias_initializer='ones')
model.add(output_layer)
print(model.summary())
```

```
model.compile(loss="categorial_crossentropy",
            optimizer="adam",
            metrics=["accuracy"])
```



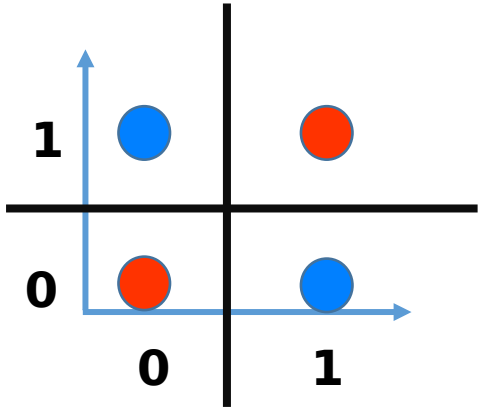

```
[array([[1., 1., 1., 1.],[1., 1., 1., 1.]],
dtype=float32),
       array([0., 0., 0., 0.], dtype=float32),
 array([[-0.03575971, -0.02119735,
0.01558626],
     [ 0.04222678, -0.04578313,  0.0360521 ],
     [-0.02753322,  0.00745941,
-0.04211374],
     [-0.04299896, -0.04190966,
0.02975568]], dtype=float32),
     array([1., 1., 1.], dtype=float32)]
```

Revenons à

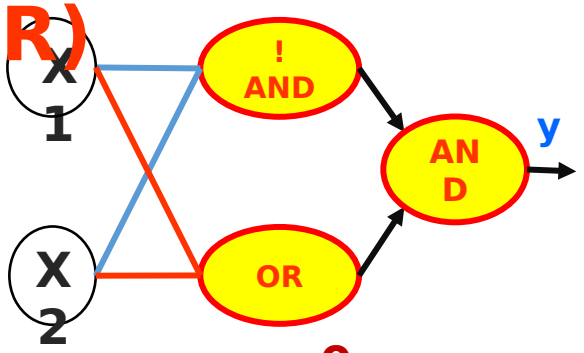
XOR Problem (simple XOR)

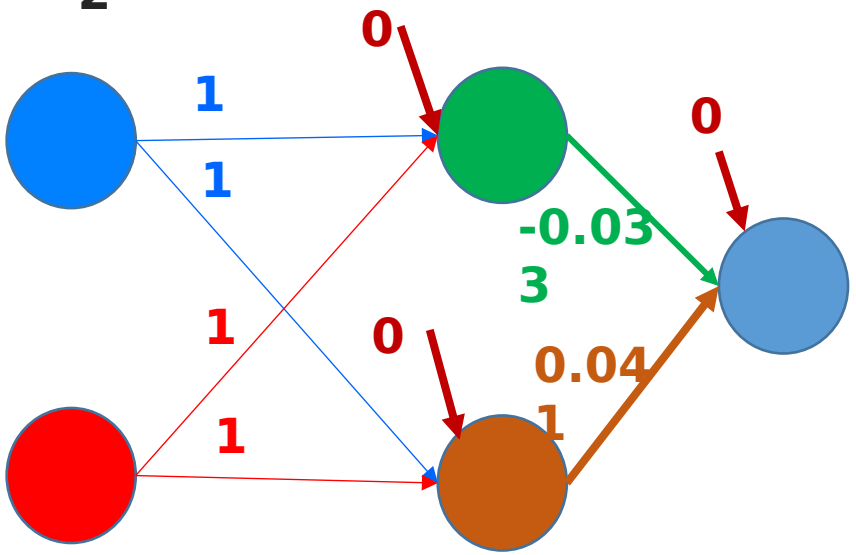
Les étapes à suivre pour créer le modèle

1. **Data**
2. **Model**
   - Create the model, compile the model: keras, tensorflow
     - Optimizer function: **sgd, adam**
     - Loss function: binary_crossentropy
   - Train the model
   - Parameters
3. **Prediction**
4. **Evaluation**



XOR = (!AND) and (OR)



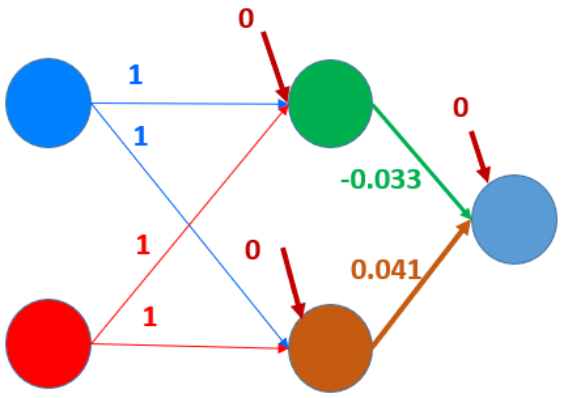


1. Dataset

```
X=np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array(np.logical_xor(X[:, 0] > 0, X[:, 1] >
0), dtype=int)
```

2. Model

```
from keras.models import Sequential

from keras.layers import Activation, Dense,Flatten

model = Sequential()

#input_layer = Flatten(input_shape=[2,1])

hidden_layer = Dense(2,input_dim=2,
kernel_initializer='ones',bias_initializer='zeros', activation='relu')

output_layer = Dense(1,kernel_initializer='random_uniform',bias_initializer='zeros',
activation='sigmoid')

#model.add(input_layer)

model.add(hidden_layer)

model.add(output_layer)

coef=model.get_weights()
```




```
#opt=tf.keras.optimizers.SGD(learning_rate=0.001)
model.compile(loss='binary_crossentropy',optimizer='sgd',metric
s=['accuracy'])
```

```
model.fit(X,y,epochs=1000)

print(model.get_weights())

print(model.predict(X)
```

[array([[1., 1.], [1., 1.]], dtype=float32), array([0., 0.], dtype=float32), array([[-0.03330588], [0.04190588]], dtype=float32), array([0.], dtype=float32)]

Optimizers

- Batch Gradient Descent
- Stochastic Gradient Descent
- Mini-Batch Gradient Descent
- Momentum Gradient Descent
- Adagrad
- RMSProp
- ADAM
- …

https://induraj2020.medium.com/different-variants-of-gradient-descent-6854d3118c4c
https://towardsdatascience.com/stochastic-gradient-descent-clearly-explained-53d239905d31
https://realpython.com/gradient-descent-algorithm-python/
https://towardsdatascience.com/7-tips-to-choose-the-best-optimizer-47bb9c1219e

Batch Gradient Descent - recall

```
def fit(X,y):
    learning_rate=0.001
    epochs=10000
    n=len(y)
    W=np.array(np.zeros(X.shape[1]))
    bias=0
    for k in range(epochs):
        yhat=bias+np.dot(X,W)
        #mean squared function as a cost function
        dbias=-(2/n)*np.sum(y-yhat)
        bias=bias-dbias* learning_rate
        dW= (2/n)* np.dot(X.T,(y-yhat))
        W=W-dW* learning_rate

    return bias,W
```

$$ms\mathcal{E} = \frac{1}{n}\sum_{i=1}^{n}(y_i - yhat_i)^2$$

Formulation used in litterature

$\theta = \theta - \alpha \cdot \nabla J(\theta)$

where:

θ is the parameter vector

α is the learning rate

$\nabla J(\theta)$ is the gradient of the cost function J with respect to θ

- Pour la mise à jour des poids, à chaque itération le dataset est utilisée entièrement
  - Le nombre de dérivés à calculer est énorme (si d est grand)
  - Convergence lente dans le cas de large

N°	X1	...	xd
1			
2			
i			
n			

Stochastic Gradient Descent

```
def fit(X,y):
    learning_rate=0.001
    epochs=10000
    n=len(y)
    W=np.array(np.zeros(X.shape[1]))
    bias=0
    for k in range(epochs):
        yhat=bias+np.dot(X,W)
        #mean squared function as a cost function
         dbias=-(2/n)*np.sum(y-yhat)
        bias=bias-dbias* learning_rate
         dW=??-(2/n)* np.dot(X.T,(y-yhat))
         W=W-dW* learning_rate

    return bias,W
```

- updates the model parameters for each training example in the dataset selected randomly
- can lead to faster convergence because the updates are more frequent and noisy
- but it can also result in more oscillations in the cost function due to the randomness of the updates.

Formulation used in litterature

$\theta = \theta - \alpha \cdot \nabla J_i(\theta)$

where:

θ is the parameter vector

α is the learning rate

$\nabla J_i(\theta)$ is the gradient of the cost function J with respect to θ computed on a single randomly selected training example i

N°	X1	...	xd
1			
2			
i			
n			

Mini-Batch Gradient Descent

```
def fit(X,y):
    learning_rate=0.001
    epochs=10000
    n=len(y)
    W=np.array(np.zeros(X.shape[1]))
    bias=0
    for k in range(epochs):
        yhat=bias+np.dot(X,W)
        #mean squared function as a cost function
        dbias=-(2/n)*np.sum(y-yhat)
        bias=bias-dbias* learning_rate
        dW=??-(2/n)* np.dot(X.T,(y-yhat))
        W=W-dW* learning_rate

    return bias,W
```

- It is a compromise between batch gradient descent and stochastic gradient descent
- ***the gradients are computed on a small random subset of the training dataset: mini-batch***
- This reduces the computational cost of the algorithm compared to batch gradient descent
- Also reducing the variance of the updates compared to SGD.
- It is widely used in deep learning because it strikes a good balance between convergence speed and stability.

Formulation used in litterature

$\theta = \theta - \alpha \cdot \frac{1}{m} \sum_{i=1}^{m} \nabla J_i(\theta)$

where:

θ is the parameter vector

α is the learning rate

m is the mini-batch size

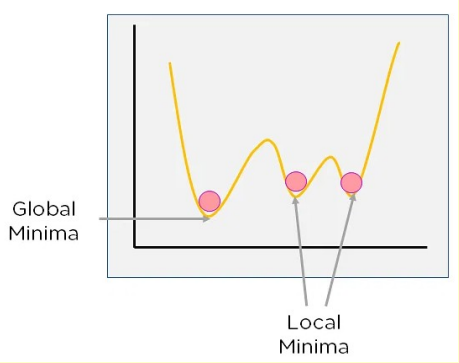
$\nabla J_i(\theta)$ is the gradient of the cost function J with respect to θ computed on a mini-batch of size m

N°	X1	...	xd
1			
2			
..			
K1			
...			
K2			
..			
n			

m

Momentum Gradient Descent

```
def fit(X,y):
    learning_rate=0.001
    epochs=10000
    n=len(y)
    W=np.array(np.zeros(X.shape[1]))
    bias=0
    for k in range(epochs):
        yhat=bias+np.dot(X,W)
        #mean squared function as a co
        dbias=-(2/n)*np.sum(y-yhat
        bias=bias-dbias* learning_ra
        dW=??-(2/n)* np.dot(X.T,(y-
        W=W-dW* learning_rate

    return bias,W
```



- ***adds a momentum term to the update rule***.
- The momentum term accumulates the gradient values over time and dampens the oscillations in the cost function, leading to faster convergence.
- This is particularly useful in cases where the cost function has a lot of noise or curvature, which can cause traditional gradient descent to get stuck in local minima.

Formulation used in litterature

$v_t = \gamma v_{t-1} + \alpha \cdot \nabla J(\theta_{t-1})$

$\theta_t = \theta_{t-1} - v_t$

where:

θ_t is the parameter vector at iteration t

α is the learning rate

$\nabla J(\theta_{t-1})$ is the gradient of the cost function J with respect to θ evaluated at θ_{t-1}

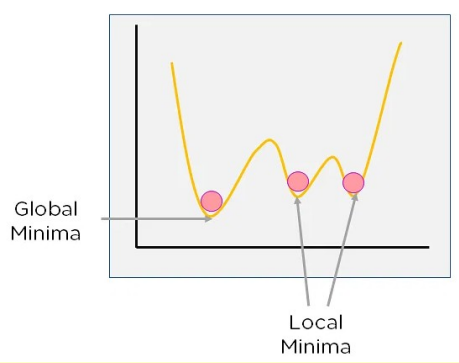
v_t is the velocity vector at iteration t

γ is the momentum coefficient, which controls the contribution of the previous velocity vector to the current velocity vector

N°	X1	...	xd
1			
2			
..			
K1			
...			
K2			
..			
n			

Adagrad

```
def fit(X,y):
    learning_rate=0.001
    epochs=10000
    n=len(y)
    W=np.array(np.zeros(X.shape[1]))
    bias=0
    for k in range(epochs):
        yhat=bias+np.dot(X,W)
        #mean squared function as a
        dbias=-(2/n)*np.sum(y-yh
        bias=bias-dbias* learning
        dW=??-(2/n)* np.dot(X.T,(
        W=W-dW* learning_rate

    return bias,W
```



- ***adapts the learning rate for each parameter based on its historical gradient values.***
- Parameters with large gradients have their learning rates reduced, while parameters with small gradients have their learning rates increased.
- This helps to normalize the updates and can be useful in cases where the cost function has a lot of curvature or different scales of gradients.

Formulation used in litterature

$$g_t = \nabla J(\theta_t)$$
$$s_t = s_{t-1} + g_t^2$$
$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{s_t+\epsilon}} \cdot g_t$$

where:

θ_t is the parameter vector at iteration t

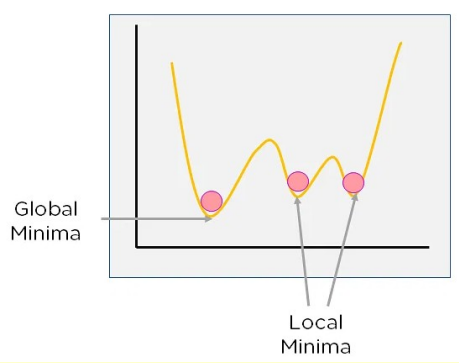
α is learning rate

$\nabla J(\theta_t)$ is gradient of the cost function J with respect to θ evaluated at θ_t

N°	X1	...	xd
1			
2			
..			
K1			
...			
K2			
..			
n			

RMSProp

```
def fit(X,y):
    learning_rate=0.001
    epochs=10000
    n=len(y)
    W=np.array(np.zeros(X.shape[1]))
    bias=0
    for k in range(epochs):
        yhat=bias+np.dot(X,W)
        #mean squared function as a
        dbias=-(2/n)*np.sum(y-yh
        bias=bias-dbias* learning
        dW=??-(2/n)* np.dot(X.T,(
        W=W-dW* learning_rate

    return bias,W
```

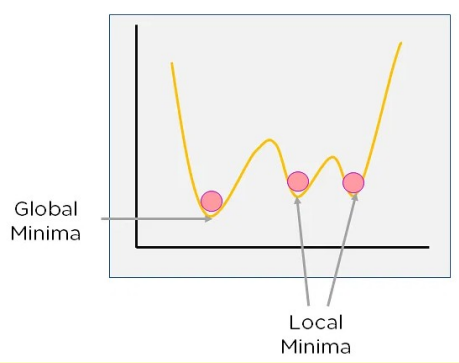


- adapts the learning rate for each parameter, but instead of using the historical gradient values, it uses a moving average of the squared gradient values
- This helps to reduce the learning rate for parameters that have large squared gradient values, which can cause the algorithm to oscillate or diverge.

N°	X1	...	xd
1			
2			
..			
K1			
...			
K2			
..			
n			

Adam

```
def fit(X,y):
    learning_rate=0.001
    epochs=10000
    n=len(y)
    W=np.array(np.zeros(X.shape[1]))
    bias=0
    for k in range(epochs):
        yhat=bias+np.dot(X,W)
        #mean squared function as a
        dbias=-(2/n)*np.sum(y-yh
        bias=bias-dbias* learning
        dW=??-(2/n)* np.dot(X.T,(
        W=W-dW* learning_rate

    return bias,W
```

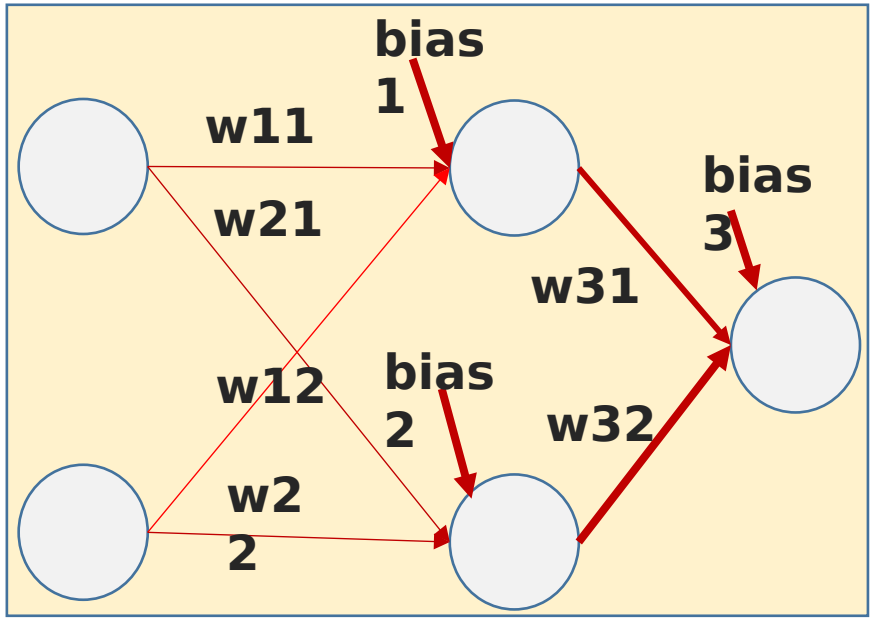


- combines the ideas of Adagrad and RMSProp.
- It adapts the learning rate for each parameter based on the historical gradient values and also uses a moving average of the gradient values to compute the momentum term.
- Adam is one of the most widely used optimization algorithms in deep learning because it is efficient, stable,
- and robust to different types of cost functions and datasets.

N°	X1	...	xd
1			
2			
..			
K1			
...			
K2			
..			
n			

Comparaisons

Variant	Advantages	Disadvantages
Batch gradient descent	Guaranteed convergence to global optimum	Computationally expensive for large datasets, slow convergence
Stochastic gradient descent	Faster convergence, more efficient for large datasets	High variance, may not converge to global optimum
Mini-batch gradient descent	Balanced convergence speed and computational cost, efficient for large datasets	Choice of mini-batch size can be a challenge
Momentum gradient descent	Faster convergence, less likely to get stuck in local minima	May overshoot and oscillate around the optimum
Adagrad	Adaptive learning rate, efficient for sparse data	Can stop learning too early
RMSProp	Adaptive learning rate, efficient for non-stationary problems	Can stop learning too early, requires tuning of hyperparameters
Adam	Adaptive learning rate, efficient for large datasets and noisy data	Can converge to suboptimal solutions, requires tuning of hyperparameters


bias
1
w11
w21
bias
3
w31
w12
bias
2
w32
w2
2

2. Model (avec une autre fonction d'activation)

```
from keras.layers import LeakyReLU
act = LeakyReLU(alpha = 0.3)
```

```
model = Sequential()
hidden_layer = Dense(2,input_dim=2, kernel_initializer='ones',bias_initializer='zeros',activation=act)
model.add(hidden_layer)
output_layer = Dense(1,kernel_initializer='random_uniform',bias_initializer='zeros',activation='sigmoid')
model.add(output_layer)
model.add(hidden_layer)
model.add(output_layer)
coef=model.get_weights()
print(coef)
```

```
model.compile(loss='binary_crossentropy',optimizer='adam',metrics=['accuracy'])
```

```
model.fit(X,y,epochs=1000)
print(model.get_weights())
print(model.predict(X))
```